Bevezetés a Rust programozásba

Gembela Gergely

7. óra: Closures, Multithreading, Concurrency

https://github.com/bmeaut/VIAUBXAV090

Closures/lambdas

- Névtelen függvények, amelyek változókban tárolhatók, és argumentumként átadhatók
- Képesek a környezetük értékeit rögzíteni (capture)
- A Rust szintaxis a Rubyéhoz hasonló

```
let clsr = |a, b| a + b;
let clsr = |a, b| {a + b};
let clsr = |a: i32, b : i32| -> i32 {a + b};
```

- Nem triviális az inferencia, az első példány típusai lesznek a típusok

Az `Fn`, `FnOnce`, és `FnMut` traitek

- `Fn`: Olyan függvények példányai számára, amelyek tetszőleges alkalommal meghívhatók (a rögzített értékek nem kerülnek „felhasználásra”)
- `FnMut`: `Fn` + mutable capture
- `FnOnce`: Olyan függvények példányai számára, amelyek egyszer hívhatók (az értékek „felhasználásra” kerülnek a hívás során)

A `move` kulcsszó

- Ha a capture lista elemeinek birtoklását át szeretnénk adni a closure-nek
  - „Capture by value” néven is találkozhatunk vele
- Multithreaded környezetben kifejezetten kényelmes

```
let moveclsr = move || {};
```

Az `Iterator` trait, és minta

- Hasonló ahhoz, ahogy C++-ban megismertétek
- Vannak műveletek, amelyek *elfogyasztják*
  - pl. `sum`, `collect`
- Vannak, amelyek egy másikat adnak vissza
  - pl. `map`, `flatten`
- Gyakran olyan closure-ökkel használjuk őket, amelyek a környezet egyéb elemeit is használják
- Hasonló lehetőségeket ad mint a Linq C#-ban

Fearless concurrency

- A Rust egyik legnagyobb marketing ígérete
- A memóriabiztonság, és a szálak közti adatmegosztás biztonsága összefüggő problémakörök
- Néhány jellemző probléma, amit meg kell oldanunk
  - Race conditions
  - Deadlock
  - Nehezen reprodukálható bugok

Többszálas programozási lehetőségek

- A Rust standard könyvtára biztosít egy 1:1 modellt implementáló többszálasítási lehetőséget
  - Emögött az operációs rendszer szálai állnak
- Lehetséges az Async szintaxis használata egy erre alkalmas runtime-mal párosítva
  - A választott runtime kezeli az async operációk futtatását

Rust threadek létrehozása

- A `thread::spawn(...);` függvénnyel indíthatunk új szálat
- Ez egy `JoinHandle`-t ad vissza
  - Ha nem csinálunk vele semmit, a program kilép a main thread befejezésekor
  - Be is tudjuk várni (`.join()`)

Move closure threadekkel

- Ha a move kulcsszóval adjuk át a closure-t egy threadnek, a thread-en futó closure lesz az átadott értékek birtokosa (ownere)
- Jellemző, hogy pl. `Arc` vagy `mpsc` sender másolatait visszük be
  - Az `Arc` és az `Rc` `downgrade()` metódusa különböző típust ad vissza!

Message passing

- Üzenetalapú szálközi kommunikáció
- Channel-eken keresztül valósul meg
  - Két részük van: küldő és fogadó
  - Akkor zárul le, ha bármely fele eldobásra kerül (dropped)

```
//multiple producer, single consumer
let (tx, rx) = mpsc::channel();
```

`mpsc`

- Küldésre a `send` használatos
- Fogadáskor vagy `recv`, vagy `try_recv`
  - Az első blokkol, amíg nem kapunk értéket
  - A második nem blokkol, a visszakapott result jelzi, hogy volt-e érték
  - Mindkettő `Result`-tal tér vissza

Send + Sync

- Marker traitek, a fordító magától „implementálja” őket
- A `Send` jelzi, hogy egy érték birtoklása átadható-e szálak közt
  - Majdnem minden típus `Send`, de pl. az `Rc<T>` nem az (a referenciaszámláló nem thread-safe)
  - Minden típus ami `Send`-típusokból épül fel, szintén `Send`

Send + Sync

- A `Sync` jelzi, hogy egy típus referenciái megoszthatók és átadhatók threadek közt
  - Ha egy típusra `&T` `Send`, akkor `T` `Sync`
  - A `RefCell` pl. nem `Sync`, mivel a runtime borrow checking logikája nem thread safe
  - Minden típus ami `Sync`-típusokból épül fel, szintén `Sync`
- A `Send` és `Sync` manuális implementációja lehetséges, de **unsafe**

Megosztott erőforrások

- Nem mindig a message passing a legkényelmesebb
- A múlt órán már említett `Arc`-ba csomagolt `Mutex` az egyik leggyakoribb minta
  - Interior mutability runtime garanciákkal

Atomics

- A Rust atomikus típusai a C++20 szabványból származnak
- Rettentő nehéz modellezni őket (250 oldalas a könyv erről)
  - A fordító műveletátrendezésére is hatással vannak
  - Hardveresen is történhet (történik) átrendezés
    - Architektúrafüggő
  - A Rustonomicon oldal tele van TODO-val a témában
- LLVM…

Scoped threads

- A `scope`-on belül indított threadek képesek referenciákra hivatkozni nem csak `'static` élettartammal
- A threadek automatikusan bevárásra kerülnek a scope végén

```
thread::scope(|s|{
  s.spawn(...);
  //...
});
```

W7T1: Táblák (második verzió)

- A második verzió célja a megoldás felkészítése többszálas környezetben történő felhasználásra
- Átgondolandó, hogy esetleg érdemes lehet-e pl. a szűrést több szálon gyorsítani

W7T2: Párhuzamos feladatok

Készíts:

- Párhuzamos prímkeresőt (+2)
- Mandelbrot-halmaz rajzolót (+3)
  - Az `image` crate-et, és scoped threadek használatát ajánlom
- Monte Carlo pi közelítést
  - Channel használatát ajánlom

Köszönöm a figyelmet!